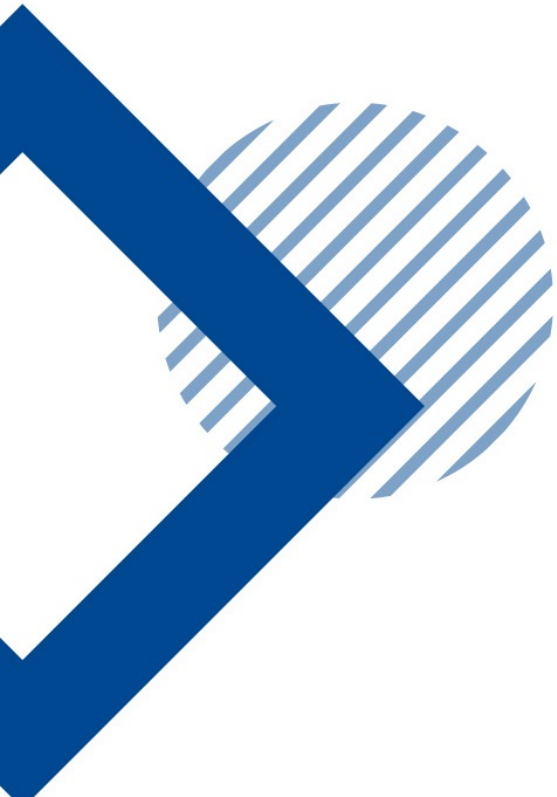
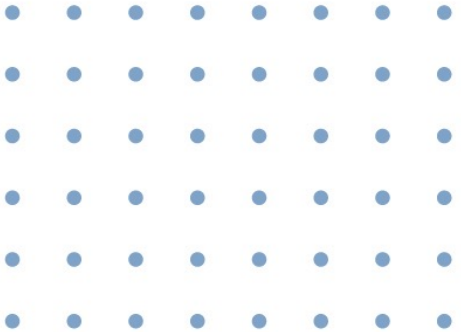
Состояние приложения

Интернет-программирование







2022

Контекст исполнения

JS хранит данные в нескольких контекстах исполнения:

1. Глобальный контекст исполнения.
2. Контекст исполнения функции.

Глобальный контекст предоставляет доступ к глобальным объектам:

```
console.log(globalThis === globalThis.globalThis); // true (everywhere)
console.log(window === window.window); // true (in a browser)
console.log(self === self.self); // true (in a browser or a Web Worker)
console.log(frames === frames.frames); // true (in a browser)
console.log(global === global.global); // true (in Node.js)
```

Переменные и функции глобального контекста являются членами глобального объекта.

Контекст исполнения функции действует только внутри соответствующей функции.

Состояние приложения 1

В некоторых случаях приложению требуется хранить свое состояние:

1. Кэширование результатов запроса к серверу.
2. Информация о текущем пользователе.
3. Временное хранение данных перед отправкой на сервер (например, корзина с покупками).
4. Отмена операций (например, онлайн редактор) и др.

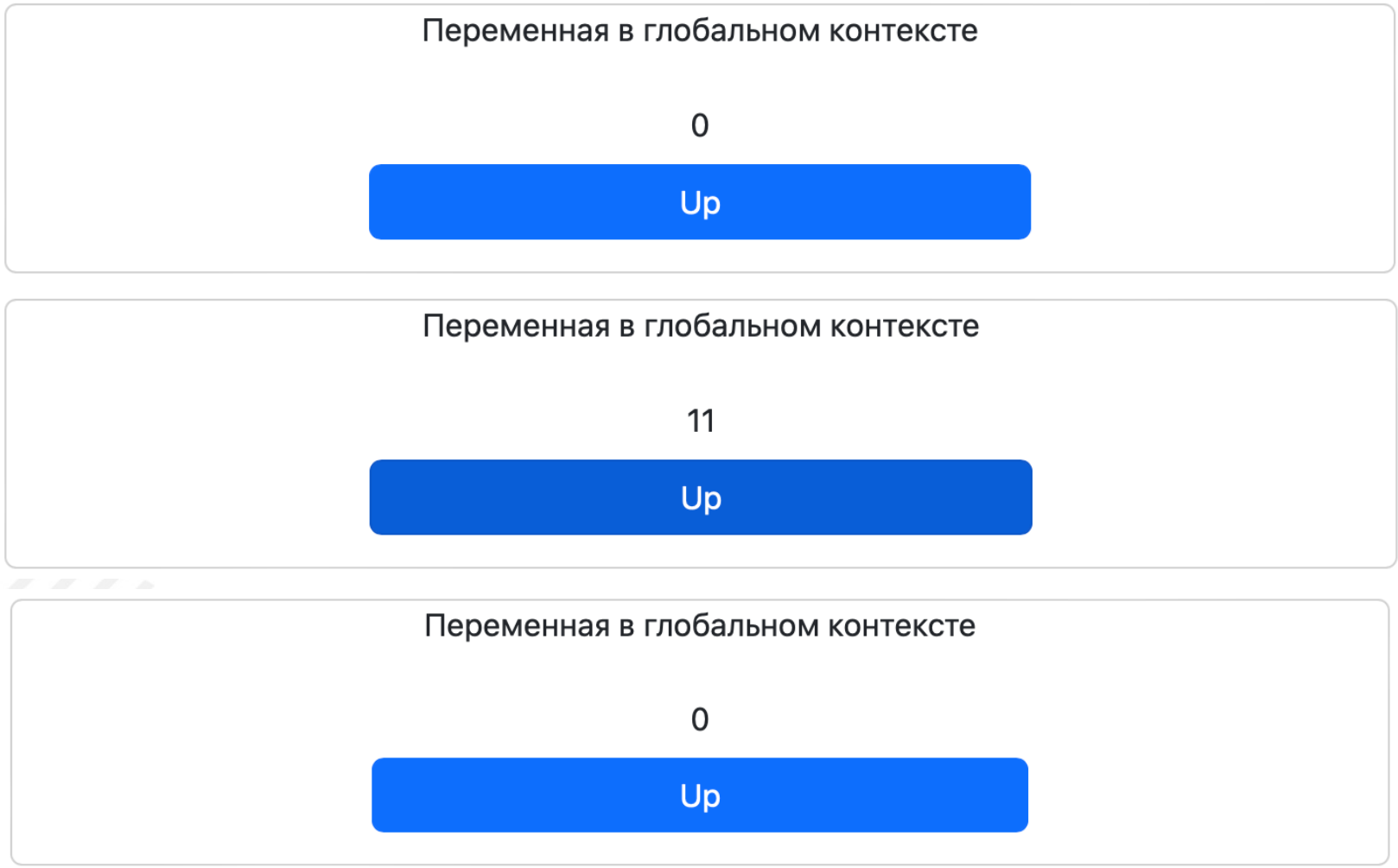
Глобальный контекст исполнения сбрасывается при обновлении страницы (НЕ маршрутизация).

Состояние приложения 2

```
<div class="card text-center mb-2">
      <div class="card-title">
         Переменная в глобальном контексте
      </div>
      <div class="card-body">
         <div class="card-text" id="gc-content"></div>
         <button class="btn btn-primary col-6 mt-2»
            onclick="updateGcContent ()">Up</button>
      </div>
</div>

const updateGcContent = (value = 0) => {
      const newState = (globalThis.myAppState || 0) + value;
      globalThis.myAppState = newState;
      document.getElementById('gc-content').innerText = newState;
}
```

Состояние приложения 3



Состояние приложения 4

Для хранения состояния можно использовать свойство localStorage глобального контекста.

Свойство localStorage позволяет работать с хранилищем Storage.

Storage представляет собой словарь вида «ключ-значение».

Для каждого сайта (origin) создается отдельный экземпляр Storage.

Для адресов http://example.com и https://example.com будут созданы собственные (разные) объекты Storage.

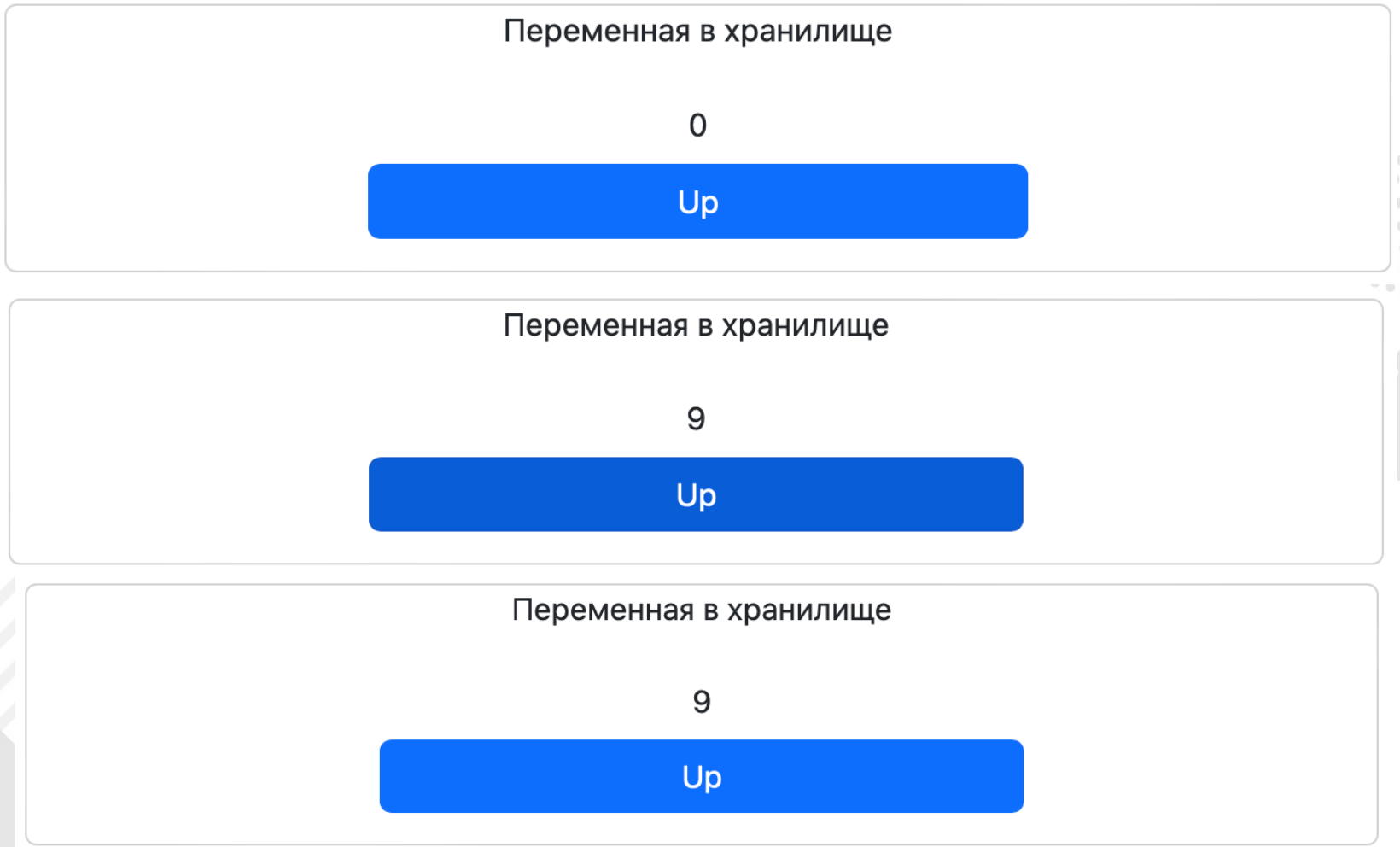
```
localStorage.setItem("myCat", "Tom");
const cat = localStorage.getItem("myCat");
localStorage.removeItem("myCat");
localStorage.clear();
```

Состояние приложения 5

```
<div class="card text-center">
      <div class="card-title">
         Переменная в хранилище
      </div>
      <div class="card-body">
         <div class="card-text" id="ls-content"></div>
         <button class="btn btn-primary col-6 mt-2"
           onclick="updateLsContent(1)">Up</button>
      </div>
 </div>

const updateLsContent = (value = 0) => {
      const lsKey = 'state';
      const newState = parseInt(localStorage.getItem(lsKey) || 0) + value;
      localStorage.setItem(lsKey, newState);
      document.getElementById('ls-content').innerText = newState;
}
```

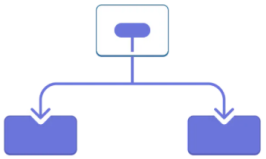
Состояние приложения 6



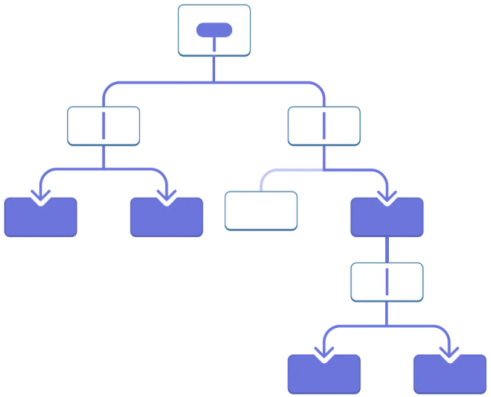
Ключ	Значение
state	9

React Context

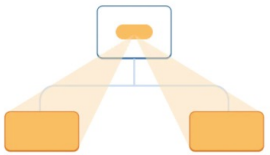
Lifting state up



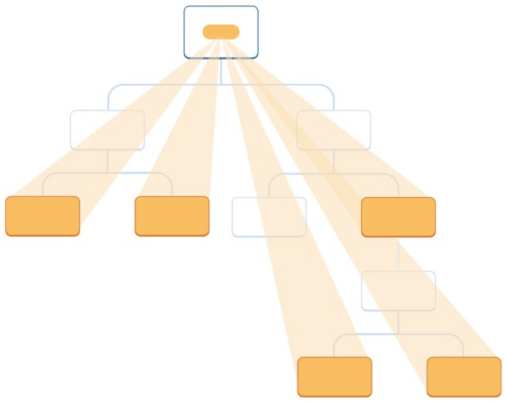
Prop drilling



Using context in close children



Using context in distant children



React Context – Создание

```
const CartContext = createContext(null);

const cart = [{...}, {...}];

export const CartProvider = ({ children }) => {
    return (
        <CartContext.Provider value={{ cart }}>
            {children}
        </CartContext.Provider>
    );
};

CartProvider.propTypes = {
    children: PropTypes.node,
};

export default CartContext;
```

React Context – Использование 1

```
const App = ({ routes }) => {
  return (
    <CartProvider>
      <Navigation routes={routes}></Navigation>
      <Container className='p-2' as='main' fluid>
        <Outlet />
      </Container>
      <Footer />
      <Toaster position='top-center' reverseOrder={true} />
    </CartProvider>
  );
};
```

React Context – Использование 2

CartHook.js

```
const useCart = () => {
 const { cart } = useContext(CartContext);
 return {
        cart,
 };
};
export default useCart;
```

Navigation.jsx

```
const { cart } = useCart();

cart.map(...)
```

React Reducer

Переход от useState к useReducer проводится в три шага:

1. Переход от состояний к отправке событий (действий).
2. Написание функции Reducer’а.
3. Использование Reducer’а в компоненте.

React Reducer – Создание 1

```
const CART_ADD = 'cart/add';
const CART_REMOVE = 'cart/remove';
const CART_CLEAR = 'cart/clear';

export const cartReducer = (cart, action) => {
    const { item } = action;
    switch (action.type) {
        case CART_ADD: {
            return addToCart(cart, item);
        }
        case CART_REMOVE: {
            return removeFromCart(cart, item);
        }
        case CART_CLEAR: {
            return [];
        }
        default: {
            throw Error(`Unknown action: ${action.type}`);
        }
    }
};
```

React Reducer – Создание 2

```
export const cartAdd = (item) => ({
    type: CART_ADD, item,
});

export const cartRemove = (item) => ({
    type: CART_REMOVE, item,
});

export const cartClear = () => ({
    type: CART_CLEAR,
});
```

React Reducer – Создание 3

```
export const CartProvider = ({ children }) => {
   const [cart, dispatch] = useReducer(cartReducer, [], loadCart);

   useEffect(() => {
      saveCart(cart || []);
   }, [cart]);

   return (
      <CartContext.Provider value={{ cart, dispatch }}>
         {children}
      </CartContext.Provider>
   );
};

export const saveCart = (cart) => {
   localStorage.setItem(CART_KEY, JSON.stringify(cart));
};

export const loadCart = (initialValue = []) => {
   const cartData = localStorage.getItem(CART_KEY);
   if (cartData) {
      return JSON.parse(cartData);
   }
   return initialValue;
};
```

React Reducer – Создание 4

```
const useCart = () => {
    const { cart, dispatch } = useContext(CartContext);

    const cartSum = () => {
        return parseFloat(
            cart?.reduce((sum, cartItem) => {
                return sum + (cartItem.price * cartItem.count);
            }, 0)
            ?? 0,
        ).toFixed(2);
    };

    return {
        cart,
        getCartSum: () => cartSum(),
        addToCart: (item) => dispatch(cartAdd(item)),
        removeFromCart: (item) => dispatch(cartRemove(item)),
        clearCart: () => dispatch(cartClear()),
    };
};
```

React Reducer – Использование 1

Lines.jsx

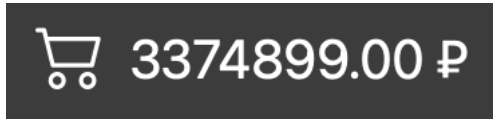
```
const { addToCart } = useCart();

<LinesTableRow key={line.id}
                    index={index} line={line}
                    onAddCart={() => addToCart(line)}
                    onDelete={() => showDeleteModal(line.id)}
                    onEdit={() => showFormModal(line.id)}
                    onEditInPage={() => showEditPage(line.id)}
                />)
```

Navigation.jsx

```
const { getCartSum } = useCart();

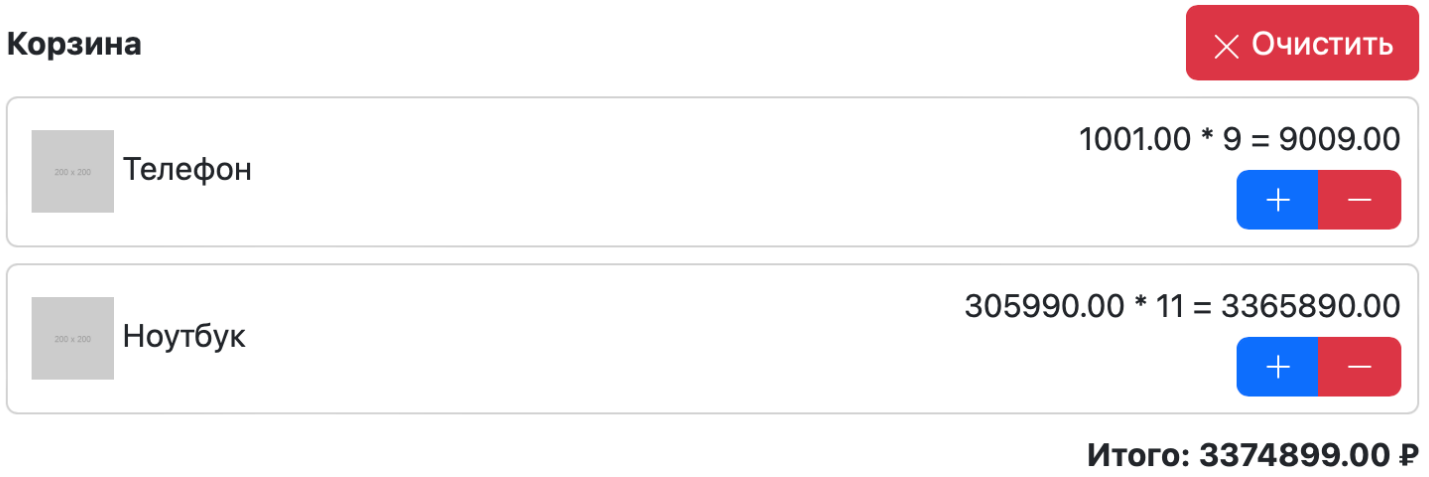
<Navbar.Brand as={Link} to='/cart'>
      <Cart2 className='d-inline-block align-top me-1 logo' /> {getCartSum() ?? ''} &#8381;
</Navbar.Brand>
```



React Reducer – Использование 2

Cart.jsx

```
const {
      cart,
      getCartSum,
      addToCart,
      removeFromCart,
      clearCart,
  } = useCart();
```




```
cart.map((cartItem) =>

<Button variant='danger' onClick={() => clearCart()}>

<Button variant="primary" onClick={() => addToCart(cartItem)}>

<Button variant="danger" onClick={() => removeFromCart(cartItem)}>

<strong>Итого: {getCartSum()} &#8381;</strong>
```

Спасибо за внимание!

Литература:

1. Global Object - https://developer.mozilla.org/en-US/docs/Glossary/Global_object
2. Global Scope - https://developer.mozilla.org/en-US/docs/Glossary/Global_scope
3. localStorage - https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage
4. JavaScript Execution Context – How JS Works Behind The Scenes - https://www.freecodecamp.org/news/execution-context-how-javascript-works-behind-the-scenes/
5. Extracting State Logic into a Reducer - https://react.dev/learn/extracting-state-logic-into-a-reducer
6. Passing Data Deeply with Context - https://react.dev/learn/passing-data-deeply-with-context
7. Scaling Up with Reducer and Context - https://react.dev/learn/scaling-up-with-reducer-and-context



2022